

ia-26/SA.py

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 import functions as f
5
6 def simulated_annealing(
7     objective_function,
8     dim=30,
9     bounds=(-100, 100),
10    max_iter=10000,
11    initial_temperature=1000,
12    cooling_rate=0.995,
13    step_size=1.0
14 ):
15     # Solução inicial
16     current_solution = np.random.uniform(
17         bounds[0],
18         bounds[1],
19         dim
20     )
21
22     current_value = objective_function(
23         current_solution
24     )
25
26     # Melhor solução global
27     best_solution = current_solution.copy()
28
29     best_value = current_value
30
31     # Temperatura inicial
32     temperature = initial_temperature
33
34     history = []
35
36     # Loop principal
37     for iteration in range(max_iter):
38
39         # Gera vizinho
40         neighbor = (
41             current_solution
42             + np.random.normal(
43                 0,
44                 step_size,
45                 dim
46             )
47         )
48
49         neighbor = np.clip(
```

```
51         neighbor,
52         bounds[0],
53         bounds[1]
54     )
55
56     neighbor_value = objective_function(
57         neighbor
58     )
59
60     # Diferença de energia
61     delta = neighbor_value - current_value
62
63     # Critério de aceitação
64     if delta < 0:
65
66         # Melhor solução → aceita
67         current_solution = neighbor
68         current_value = neighbor_value
69
70     else:
71
72         # Aceita solução pior
73         probability = np.exp(
74             -delta / temperature
75         )
76
77         if np.random.rand() < probability:
78
79             current_solution = neighbor
80             current_value = neighbor_value
81
82     # Atualiza melhor global
83     if current_value < best_value:
84
85         best_solution = (
86             current_solution.copy()
87         )
88
89         best_value = current_value
90
91     # Resfriamento
92     temperature *= cooling_rate
93
94     history.append(best_value)
95
96     print(
97         f"Iteração {iteration+1:5d}"
98         f" | Melhor valor: "
99         f"{best_value:.10f}"
100        f" | Temperatura: "
101        f"{temperature:.5f}"
102    )
103
```

```
104     return (
105         best_solution,
106         best_value,
107         history
108     )
109
110 def plot_convergence(history):
111
112     plt.figure(figsize=(8, 5))
113
114     plt.plot(history)
115
116     plt.title("Convergência do Simulated Annealing")
117
118     plt.xlabel("Iteração")
119     plt.ylabel("Melhor valor")
120
121     plt.yscale("log")
122
123     plt.grid(True)
124
125     plt.show()
126
127
128 if __name__ == "__main__":
129
130     objective_function = f.f1
131
132     bounds = (-100, 100)
133
134     best_position, best_value, history = (
135         simulated_annealing(
136             objective_function=objective_function,
137             dim=30,
138             bounds=bounds,
139             max_iter=10000,
140             initial_temperature=1000,
141             cooling_rate=0.995,
142             step_size=1.0
143         )
144     )
145
146     plot_convergence(history)
147
148     print("\n===== RESULTADO FINAL =====")
149
150     print("Melhor posição:")
151     print(best_position)
152
153     print("\nMelhor valor:")
154     print(best_value)
```